

BOĞAZIÇI UNIVERSITY

**INSTITUTE FOR GRADUATE STUDIES IN SCIENCE AND
ENGINEERING**

**PERFORMANCE EVALUATIONS OF
LLM INFERENCE FRAMEWORKS**

Final Report

CMPE 492 — Senior Design Project

**Eray Yüklü
Süleyman Tolga Acar**

**Advisor:
Bahri Atay Özgövde**

2026

**<https://github.com/erayyuklu/>
PERFORMANCE-EVALUATIONS-OF-LLM-INFERENCE-FRAMEWORKS**

Contents

1	INTRODUCTION	1
1.1	Motivation and Background	1
1.2	Project Goals	2
1.3	Broad Impact	3
1.4	Ethical Considerations	3
2	PROJECT DEFINITION AND PLANNING	4
2.1	Project Definition	4
2.2	Project Planning	5
2.2.1	Project Time and Resource Estimation	5
2.2.2	Success Criteria	6
2.2.3	Risk Analysis	7
2.2.4	Team Work	7
3	RELATED WORK	8
3.1	LLM Inference Engines	8
3.2	KV Cache Optimization	8
3.3	Quantization for Inference	9
3.4	LLM Serving Benchmarks	9
3.5	Reasoning Models	9
3.6	KV Cache Quantization	10
3.7	Agentic LLM Systems and Cost Routing	10
3.8	Agent Benchmarks: GAIA	10
4	METHODOLOGY	12
4.1	Metrics Framework	12
4.1.1	Latency Metrics	12
4.1.2	Throughput Metrics	13
4.1.3	Resource Utilization Metrics	13
4.1.4	Quality Metrics	13
4.1.5	Agent Evaluation Metrics (Phase 2)	13

4.2	Statistical Reporting	14
4.3	Experimental Hypotheses	14
5	REQUIREMENTS SPECIFICATION	16
5.1	Functional Requirements	16
5.2	Non-Functional Requirements	16
5.3	Use Cases	17
6	DESIGN	18
6.1	System Architecture	18
6.1.1	Infrastructure	18
6.1.2	Load Generator (Locust)	18
6.1.3	Experiment Runner	19
6.1.4	Telemetry Collection	19
6.1.5	Analysis Pipeline	20
6.1.6	Agent Application (Phase 2)	20
6.2	Information Structure	21
6.3	Information Flow	21
6.4	Deployment Architecture	22
6.4.1	GKE Deployment	22
6.4.2	Lightning AI Deployment	22
6.5	User Interface Design	22
7	IMPLEMENTATION AND TESTING	24
7.1	Implementation	24
7.1.1	vLLM Deployment Scripts	24
7.1.2	Locust Load Generator	24
7.1.3	Experiment Runner	25
7.1.4	Budget Guard	25
7.1.5	Quality Evaluation Pipeline	25
7.1.6	Phase 2 Agent Application	25
7.1.7	GAIA Evaluation Harness	26
7.2	Testing	26
7.2.1	Unit-Level Verification	26
7.2.2	Integration Testing	27
7.2.3	Experiment Validation	27
7.3	Deployment	27
7.3.1	GKE Deployment Procedure	27
7.3.2	Lightning AI Deployment Procedure	27

8	RESULTS	28
Part I — Phase 1: Serving Optimization		28
8.1 Pilot Study: vLLM Server Parameter Sweep		28
8.1.1 Baseline Performance		28
8.1.2 Effect of max-num-seqs		29
8.1.3 Effect of gpu-memory-utilization		29
8.1.4 Effect of Chunked Prefill		29
8.1.5 Effect of Prefix Caching		29
8.1.6 Throughput and Cost in the Pilot		30
8.2 Main Campaign: Quantization — Reasoning Model (DeepSeek-R1)		30
8.2.1 FP8 Quantization		30
8.2.2 BitsAndBytes (INT4) Quantization		31
8.3 Main Campaign: Quantization — Non-Reasoning Model (Qwen2.5-7B-Instruct)		31
8.4 Reasoning vs. Non-Reasoning Model Comparison		31
8.5 Main Campaign: I/O Profile Analysis		32
8.5.1 TTFT by Profile		32
8.5.2 E2E Latency by Profile		32
8.6 Throughput and Cost Analysis		32
8.7 Quality Evaluation		33
8.8 Data Completeness		35
8.9 Extended Study: Qwen3-8B Reasoning vs. Non-Reasoning		35
8.10 KV Cache Precision: Default vs. FP8 (Qwen3-8B)		36
8.11 TurboQuant KV-Cache Compression		38
8.12 Model Comparison: Gemma 4 vs. Qwen3-8B		39
Part II — Phase 2: Asymmetric Agent on GAIA		41
9	CONCLUSION	43
9.1 Summary of Findings		43
9.2 Practical Recommendations		45
9.3 Limitations		45
9.4 Future Work		46
A	vLLM PARAMETER REFERENCE	49
A.1 Install-Time Parameters		49
A.2 Key Runtime Parameters (vllm serve)		49
B	EXPERIMENT CONFIGURATIONS	51
B.1 Server Tuning Experiments (Pilot)		51
B.2 Quantization Experiments		51

B.3	KV Cache and TurboQuant Experiments (Qwen3-8B)	52
B.4	Agent Configurations (Phase 2)	52
C	DATASET DESCRIPTIONS	53
C.1	Custom Prompt Dataset	53
C.2	Prefix Caching Dataset	53
C.3	Quality Evaluation Datasets	53
C.4	GAIA Benchmark (Phase 2)	54

Chapter 1

INTRODUCTION

1.1 Motivation and Background

Large Language Models (LLMs) have rapidly become central to a wide range of applications, from interactive chatbots and code assistants to scientific research tools and enterprise automation platforms. As demand for LLM-powered services grows, the cost and latency of serving these models at scale have become critical engineering concerns. A single inference request to a modern 7-billion-parameter model can consume hundreds of milliseconds of GPU compute time, and serving thousands of concurrent users demands careful optimization of both hardware utilization and serving software configuration.

vLLM [9] is an open-source, high-throughput inference engine purpose-built for LLMs. It introduced PagedAttention, a memory management technique inspired by operating system virtual memory paging, which dramatically reduces GPU memory fragmentation and enables significantly more concurrent requests per GPU compared to earlier approaches. Beyond memory management, vLLM exposes a rich set of tunable parameters governing scheduling, batching, quantization, and caching—each of which can substantially alter the throughput–latency trade-off of the serving system.

Despite vLLM’s widespread adoption, systematic empirical studies that isolate and quantify the effect of individual configuration parameters under realistic multi-user loads remain relatively scarce. Most published benchmarks focus on throughput at a single concurrency level, or on quantization accuracy, but do not jointly analyze latency distributions, cost efficiency, GPU resource utilization, and model quality across a comprehensive parameter matrix.

A second, equally pressing concern emerges one level above the serving engine: the application architecture. Deploying advanced agentic workflows—systems in which an LLM autonomously plans, calls tools, and executes multi-step tasks—at scale using commercial LLM APIs is financially prohibitive, while budget-friendly self-hosted

models often lack the reasoning capacity for complex tasks. Bridging this gap requires both an optimized serving layer *and* an architecture that allocates expensive reasoning capacity only where it is genuinely needed.

This project addresses both gaps through a two-phase study. **Phase 1** optimizes self-hosted LLM serving on a single NVIDIA L4 GPU through systematic vLLM parameter sweeps, evaluating latency, throughput, and memory metrics alongside standard quality benchmarks. **Phase 2** then implements a hybrid *Planner-Executor* agent architecture that combines the planning strengths of a commercial API with the low cost of the optimized self-hosted execution layer to achieve cost-efficient, high-accuracy task resolution. Together the two phases form a single narrative: Phase 1 establishes how cheaply and how fast a self-hosted model can be served, and Phase 2 exploits that optimized serving layer inside a real agentic application benchmarked on GAIA.

1.2 Project Goals

The project is organized into two complementary phases. The primary goals are:

Phase 1 — Serving Optimization:

- (i) Deploy vLLM on cloud GPU infrastructure (Google Kubernetes Engine and Lightning AI) in a reproducible and automatable manner.
- (ii) Design a comprehensive set of benchmark experiments covering vLLM server-side parameters, model-weight quantization (FP16, FP8, AWQ/INT4), KV cache precision formats (FP16, FP8, and TurboQuant levels), model types, and input/output profiles.
- (iii) Measure a rich set of latency, throughput, resource utilization, and model quality metrics for each configuration.
- (iv) Identify which configurations lie on the Pareto frontier of the throughput–latency–cost–quality trade-off space.

Phase 2 — Asymmetric Agent:

- (v) Implement a hybrid Planner–Executor agent that routes high-level reasoning to a commercial API (the Planner) and bulk execution to the optimized self-hosted model (the Executor) deployed in Phase 1.
- (vi) Benchmark five agent configurations on the GAIA Level 1 validation set, measuring accuracy, end-to-end task duration, and API cost per question.
- (vii) Locate the Pareto-optimal agent architecture in the accuracy-versus-cost space.

- (viii) Produce actionable recommendations for practitioners deploying both LLM serving stacks and agentic applications on commodity cloud GPUs.

1.3 Broad Impact

The findings of this project have direct practical relevance for any organization deploying LLMs on cloud infrastructure. By quantifying the precise trade-offs associated with FP8 quantization, INT4 quantization, chunked prefill, prefix caching, and batch-size tuning, this work helps practitioners make informed decisions that can reduce serving costs by up to $7\times$ without sacrificing response quality. Reducing GPU energy consumption and cloud costs per inference request also has positive environmental and economic implications as LLM serving continues to scale globally.

Furthermore, the open-source benchmarking pipeline developed in this project—comprising a parameterized experiment runner, a Locust-based load generator with SSE streaming support, GPU and vLLM telemetry collectors, and a Jupyter analysis notebook—can be reused and extended by the research community for evaluating other inference engines, models, and hardware platforms.

Phase 2 extends this impact to the application layer. By showing that a hybrid Planner–Executor agent can match roughly 80% of a fully-commercial agent’s GAIA accuracy at roughly one-tenth of the cost, the project gives practitioners a concrete blueprint for building affordable agentic systems: spend commercial-API budget only on planning, and offload the high-volume execution work to a self-hosted, FP8-optimized model.

1.4 Ethical Considerations

The project uses only publicly available model weights (DeepSeek-R1-Distill-Qwen-7B, Qwen2.5-7B-Instruct, Qwen3-8B, and Gemma 4, all under open research licenses) and publicly available benchmark datasets (ShareGPT, ARC-Challenge, MMLU, GSM8K, and the GAIA validation set). No private user data was collected or processed. All experiments were conducted on cloud infrastructure with standard access controls. Cost overruns were guarded against by a GCP budget alert system with an automatic billing-disable Cloud Function. The evaluation benchmarks (ARC-Challenge, MMLU, GSM8K, GAIA) are well-established academic datasets, and no benchmark contamination was introduced because the models were not fine-tuned during the study. The Phase 2 agent executes model-generated Python code only inside a sandboxed cloud environment (E2B), preventing untrusted code from touching project infrastructure.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

This project systematically evaluates the performance of vLLM, an open-source LLM inference engine, across a range of configuration parameters and model variants, and then applies the optimized serving layer inside a cost-efficient agentic application. The study spans two phases.

Phase 1 (Serving Optimization) centers on a progression of self-hosted 7–8B models—a reasoning model (DeepSeek-R1-Distill-Qwen-7B), a non-reasoning model (Qwen2.5-7B-Instruct), the dual-mode Qwen3-8B, and Google’s Gemma 4 family—served on NVIDIA L4 GPUs (24 GB VRAM) hosted on Google Cloud Platform (GCP) and Lightning AI. Across these models the project sweeps model-weight quantization (FP16, FP8, AWQ/INT4), KV cache precision (default/FP16, FP8, and TurboQuant levels), scheduler and memory parameters, and input/output profiles.

Phase 2 (Asymmetric Agent) builds a ReAct-style LangGraph agent on top of the Phase 1 serving stack. A high-capability commercial model (Gemini 3 Flash) acts as the Planner, decomposing a user query into a structured plan; a cost-efficient self-hosted model (an AWQ-quantized Gemma 4 26B served by vLLM) acts as the Executor, carrying out the plan with a suite of real tools (web search, web scraping, document parsing, and sandboxed Python execution). Five Planner/Executor combinations are benchmarked on the GAIA Level 1 validation set.

The benchmarking pipeline is fully automated: a shell-script experiment runner iterates over a JSON-defined experiment matrix, restarts vLLM with new parameters for each experiment, triggers Locust load tests, collects per-request latency metrics via Server-Sent Events (SSE) parsing, captures GPU and vLLM telemetry, and stores all results as CSV files for offline analysis. A Jupyter notebook then performs statistical

analysis and produces plots covering latency distributions, throughput, cost efficiency, GPU utilization, KV cache behavior, and model quality.

The principal experiment campaigns are:

- (i) **Server Tuning (Pilot)**: A preliminary sweep of vLLM scheduler parameters including `max-num-seqs`, `gpu-memory-utilization`, `enable-chunked-prefill`, and `enable-prefix-caching`.
- (ii) **Weight Quantization — Reasoning Model**: FP16 baseline, FP8, and BitsAndBytes (INT4) quantization on DeepSeek-R1.
- (iii) **Weight Quantization — Non-Reasoning Model**: FP16 baseline and FP8 on Qwen2.5-7B-Instruct.
- (iv) **I/O Profiles**: Six prompt categories (all, coding-only, creative-only, reasoning-only, short-input, long-input) to isolate the effect of input/output length on latency.
- (v) **Reasoning vs. Non-Reasoning (Qwen3-8B)**: The dual-mode Qwen3-8B with `enable_thinking` on and off under identical load.
- (vi) **KV Cache Precision**: Default (auto) vs. FP8 KV cache on Qwen3-8B, scaling to 128 concurrent users.
- (vii) **TurboQuant KV Cache**: Five KV-cache precision presets (FP8 baseline through 3-bit keys + 3-bit values) on Qwen3-8B.
- (viii) **Model Comparison**: Gemma 4 E4B vs. Qwen3-8B at matched FP8 KV configuration.
- (ix) **Agent Architecture (Phase 2)**: Five Planner/Executor configurations evaluated on GAIA Level 1.

2.2 Project Planning

2.2.1 Project Time and Resource Estimation

The project spanned two academic terms. Phase 1 occupied the first term (CMPE 491) and Phase 2 the second (CMPE 492), proceeding in four major stages:

- **Weeks 1–3**: Infrastructure setup (GKE cluster, vLLM deployment, Locust deployment, Prometheus + Grafana monitoring). Literature review of vLLM, PageAttention, quantization methods, and LLM serving benchmarks.

- **Weeks 4–8:** Pilot experiments on Lightning AI (server tuning parameter sweep, prefix caching study). Development and refinement of the automated experiment runner, Locust streaming client, GPU monitor, and analysis notebook.
- **Weeks 9–14:** Full Phase 1 weight-quantization and I/O-profile campaigns on GCP, followed by the extended Phase 1 study on Qwen3-8B (reasoning vs. non-reasoning, FP8 KV cache, TurboQuant KV-cache levels) and the Gemma 4 vs. Qwen3 model comparison. Quality evaluation using lm-evaluation-harness.
- **Weeks 15–24:** Phase 2 agent development (LangGraph Planner–Executor architecture, real tool suite, Postgres checkpointing, Langfuse observability), GAIA Level 1 evaluation across five agent configurations, cost analysis, poster preparation, and final report writing.

GPU compute costs were managed through a GCP budget alert (12,500 TRY limit) connected to a Pub/Sub topic and a Cloud Function that automatically disables billing when the threshold is reached.

2.2.2 Success Criteria

The project was considered successful upon meeting the following criteria:

- (i) A fully automated, reproducible experiment pipeline capable of running any experiment defined in a JSON configuration file.
- (ii) At least 2,000 successfully processed benchmark requests with a near-zero error rate.
- (iii) Quantitative characterization of at least three vLLM parameters across at least two concurrency levels.
- (iv) Quality evaluation results (lm-evaluation-harness scores) for all quantization variants.
- (v) A unified analysis notebook producing publication-quality plots covering all metric categories.
- (vi) A working Planner–Executor agent benchmarked on GAIA Level 1, with at least one hybrid configuration demonstrating a substantial cost reduction relative to the all-commercial baseline at comparable accuracy.

All criteria were met. The Phase 1 weight-quantization campaign processed 2,041 requests at 100% success rate; the Qwen3-8B campaigns added 2,274 requests (99.7% success) and the TurboQuant campaign a further 1,093 requests. In Phase 2, the hybrid

Planner–Executor agent reached 67.9% GAIA Level 1 accuracy at roughly one-tenth the cost of the all-commercial baseline (84.9%).

2.2.3 Risk Analysis

Table 2.1: Risk analysis for the project.

Risk	Probability	Impact	Mitigation
GCP GPU quota unavailable	High	High	Fallback to Lightning AI; budget alert to prevent cost overruns
vLLM version API changes	Medium	Medium	Pin vLLM version; use YAML config abstraction
OOM errors under high concurrency	Medium	Medium	Monitor KV cache; limit max-num-seqs
Locust worker crashes during long tests	Low	Medium	Use distributed master/worker Locust setup; retry logic
Model download failures (HuggingFace)	Low	Low	Cache model weights on persistent disk

2.2.4 Team Work

The project was conducted by two students. Infrastructure setup, Kubernetes manifests, and the GCP/Lightning AI deployment scripts were developed collaboratively. Eray Yüklü led the development of the Locust load generator, SSE streaming parser, and the per-request metrics collection pipeline. Süleyman Tolga Acar led the experiment runner automation, GPU/vLLM telemetry collection, and the Jupyter analysis notebook. Both students jointly designed the experiment matrix, ran the experiments, and contributed to the analysis and report writing. In Phase 2, the LangGraph agent (graph definition, planner/executor modes, real-tool suite, Postgres checkpointing, and Langfuse observability) and the GAIA evaluation harness were likewise developed jointly, with the agent load-testing and cost-accounting work building directly on the Phase 1 telemetry pipeline.

Chapter 3

RELATED WORK

3.1 LLM Inference Engines

The field of LLM inference optimization has grown rapidly in parallel with the proliferation of large transformer models. Early serving systems such as TensorFlow Serving and TorchServe were designed for classical deep learning workloads and lack the specialized memory management required for autoregressive generation.

vLLM [9] introduced PagedAttention, which allocates the KV cache in fixed-size blocks analogous to OS memory pages, nearly eliminating fragmentation and enabling 2–4× higher throughput than systems using contiguous KV cache allocation. Concurrent work includes TensorRT-LLM [13], which uses NVIDIA’s TensorRT compiler to produce highly optimized CUDA kernels; NVIDIA Dynamo (TorchDynamo-based); and Hugging Face Text Generation Inference (TGI). Each system offers different trade-offs between portability, ease of use, and peak throughput.

vLLM’s V1 engine (version 0.16+), used in this project, introduced further improvements including a redesigned scheduler, native support for chunked prefill as a default, and improved prefix caching.

3.2 KV Cache Optimization

The KV (Key-Value) cache stores the intermediate attention states of already-processed tokens, enabling efficient incremental decoding. Efficient KV cache management is critical because its memory footprint grows linearly with context length, number of transformer layers, and batch size.

PagedAttention [9] addresses fragmentation. Prefix caching [18] extends this by reusing KV blocks for tokens that are shared across requests—most commonly a system prompt—reducing the TTFT for repeated-prefix workloads. RadixAttention [17] generalizes this to a trie structure for more flexible prefix reuse. This project empiri-

cally measures the benefit of vLLM’s built-in prefix caching implementation.

3.3 Quantization for Inference

Model quantization reduces the numerical precision of weights and activations, decreasing both memory footprint and, on hardware with native low-precision support, computation time. FP8 quantization is natively accelerated on NVIDIA H100 and, partially, on L4 GPUs. AWQ [11] and GPTQ [7] are popular 4-bit weight-only quantization schemes that apply calibration-based rounding to minimize accuracy loss. BitsAndBytes [6] provides a library-level INT4/INT8 quantization compatible with any CUDA GPU.

Prior work [5] demonstrates that INT8 quantization introduces negligible accuracy loss for most tasks, while INT4 methods show greater variance. This project extends these findings to the specific context of LLM serving under concurrent load, measuring both the latency impact and the quality impact across multiple benchmarks. The Phase 2 Executor uses an AWQ 4-bit quantized Gemma 4 26B model, chosen because AWQ’s calibration-based weight-only scheme preserves accuracy well while shrinking the model enough to serve comfortably on a single L4.

3.4 LLM Serving Benchmarks

The vLLM benchmark tool (included in the vLLM repository) and ShareGPT-based datasets are standard for measuring serving throughput. Agrawal et al. [1] study the prefill-decode interference problem and propose chunked prefill as a mitigation. The LMSys Chatbot Arena [3] provides production traffic traces. MLPerf Inference [15] defines standardized offline and server scenarios for LLM benchmarking.

This project uses Locust as the load generator rather than the vLLM benchmark tool, which enables more realistic concurrent user simulation, SSE streaming support, and per-request metric collection (TTFT, TPOT, ITL) that the vLLM benchmark tool does not natively provide.

3.5 Reasoning Models

DeepSeek-R1 [4] is a reasoning-capable model that generates a chain-of-thought `<think>...</think>` block before producing its final answer. This significantly increases the number of output tokens per request compared to a non-reasoning model answering the same question directly, with corresponding effects on E2E latency and throughput. This project is among the first to systematically compare the serving performance of reasoning and

non-reasoning 7B models under identical load conditions. The later Qwen3-8B [14] model exposes both modes through a single `enable_thinking` switch, allowing a clean within-model comparison.

3.6 KV Cache Quantization

Beyond quantizing model weights, the KV cache itself can be stored at reduced precision. vLLM supports an FP8 KV cache (`-kv-cache-dtype fp8`) that halves the per-token KV footprint relative to the default 16-bit cache, allowing more concurrent sequences to fit in a fixed VRAM budget. More aggressive schemes such as TurboQuant compress keys and values to 4-bit or even 3-bit representations with optional norm correction. The central trade-off is between memory savings (which can raise effective concurrency) and the per-token dequantization compute added to the decode path. On GPUs where the KV cache dominates VRAM, this trade-off favours compression; on smaller models or compute-bound hardware it can invert. This project measures that trade-off directly on the L4 for an 8B model.

3.7 Agentic LLM Systems and Cost Routing

Agentic systems wrap an LLM in a loop that interleaves reasoning with external tool calls. The ReAct paradigm [16] formalises this as alternating “thought” and “action” steps, and frameworks such as LangGraph [10] provide stateful graph abstractions for building such agents in production. A recurring concern is cost: routing every reasoning and execution step to a frontier commercial API is expensive at scale. FrugalGPT [2] showed that cascading and routing queries across models of different costs can cut API spend dramatically while preserving accuracy. The Planner–Executor (or “big brain, small hands”) pattern is a specific instance of this idea: a strong, expensive model produces a plan, and a cheaper model carries it out. Phase 2 of this project realises this pattern with a commercial Planner (Gemini 3 Flash) and a self-hosted, FP8/AWQ-optimised Executor served by the Phase 1 vLLM stack, and quantifies the resulting accuracy-versus-cost trade-off.

3.8 Agent Benchmarks: GAIA

GAIA [12] is a benchmark of real-world questions for general AI assistants that require multi-step reasoning, web browsing, file handling, and code execution to answer. Unlike knowledge benchmarks such as MMLU, GAIA questions have unambiguous, exact-match answers but demand correct tool orchestration to reach them, making the

benchmark a faithful proxy for agentic capability. Phase 2 uses the GAIA Level 1 validation split (53 questions) to compare agent architectures on accuracy, task duration, and cost.

Chapter 4

METHODOLOGY

4.1 Metrics Framework

We measure a comprehensive set of metrics drawn from three sources: client-side (Locust), server-side (vLLM Prometheus endpoint), and GPU-level (nvidia-smi).

4.1.1 Latency Metrics

Time to First Token (TTFT) is the wall-clock time from the moment the HTTP request is dispatched until the first output token arrives at the client. It captures the prefill phase duration, any queuing delay, and network round-trip overhead. TTFT is the primary indicator of perceived responsiveness.

Time Per Output Token (TPOT) is the average time between consecutive output tokens during the decode phase, computed as:

$$\text{TPOT} = \frac{t_{\text{last token}} - t_{\text{first token}}}{\max(N_{\text{output}} - 1, 1)} \quad (4.1)$$

TPOT determines how fast text streams to the user. A TPOT exceeding approximately 60 ms/token corresponds to slower than comfortable reading speed.

End-to-End Latency (E2E) is the total wall-clock time from request submission to the final token:

$$\text{E2E} = \text{TTFT} + \text{TPOT} \times (N_{\text{output}} - 1) \quad (4.2)$$

Inter-Token Latency (ITL) measures the individual time gaps between each pair of consecutive tokens. Unlike TPOT (which is an average), ITL variance exposes jitter—a “stuttering” stream that users perceive as laggy even if the mean is acceptable. We report ITL as P50 and P99 per request.

4.1.2 Throughput Metrics

Output Token Throughput (tokens/s) is the total number of output tokens generated across all concurrent requests per unit time:

$$\text{Throughput} = \frac{\sum N_{\text{output}}}{T_{\text{test duration}}} \quad (4.3)$$

Cost per Million Output Tokens (USD/M tokens) combines throughput with the GPU hourly cost:

$$\text{Cost/M} = \frac{\text{GPU rate (USD/hr)} \times T_{\text{hours}}}{\sum N_{\text{output}}/10^6} \quad (4.4)$$

4.1.3 Resource Utilization Metrics

GPU utilization percentage, VRAM usage (MB), GPU power draw (W), and temperature (°C) are sampled every 2 seconds via `nvidia-smi`.

KV cache utilization percentage (`vllm:kv_cache_usage_perc`), number of running and waiting requests (`vllm:num_requests_running`, `vllm:num_requests_waiting`), and generation throughput (`vllm:generation_tokens_total`) are scraped from the vLLM Prometheus endpoint every 2 seconds.

4.1.4 Quality Metrics

Model quality is evaluated offline using `lm-evaluation-harness` [8] on three standard benchmarks: ARC-Challenge (5-shot, normalized accuracy), MMLU (5-shot, accuracy), and GSM8K (5-shot, exact match). Quality evaluations are run after each quantization variant is deployed to the same vLLM endpoint used for load testing, ensuring consistency.

4.1.5 Agent Evaluation Metrics (Phase 2)

The Phase 2 agent is evaluated on the GAIA Level 1 validation set (53 questions) using three metrics per configuration:

Accuracy is the fraction of questions answered correctly under GAIA’s exact-match scorer, which normalizes case and whitespace, strips currency/unit symbols, and canonicalizes numbers before comparison.

Task Duration is the mean wall-clock time (seconds) for the agent to resolve a single question end-to-end, including all planner calls, executor calls, and tool invocations.

Cost per Question combines commercial-API token usage (priced at the Gemini 3 Flash published rate) with self-hosted token usage (priced at the amortized L4 GPU

rate derived from Phase 1 throughput). We report both a raw cost (charging only for billable commercial-API tokens) and a *cost-adjusted* figure that additionally imputes a price to self-hosted Executor tokens, so that fully self-hosted configurations are not credited as free.

4.2 Statistical Reporting

Raw averages can be misleading for latency distributions, which are typically right-skewed. All latency metrics are reported as P50 (median), P95, and P99 in addition to the mean. P99 is of particular operational significance: with 1,000 requests per minute, a P99 of 5 seconds means roughly 10 users per minute experience a 5-second wait—a direct SLA concern.

4.3 Experimental Hypotheses

Before running experiments, the following hypotheses were formulated:

- FP8 quantization will reduce TTFT and TPOT by 10–30% with less than 3% quality degradation.
- BitsAndBytes (INT4) quantization will further reduce memory footprint but may degrade latency due to dequantization overhead on the L4 GPU.
- The reasoning model will have comparable TTFT but much higher E2E latency than the non-reasoning model due to chain-of-thought output.
- Prefix caching will reduce TTFT by more than 50% for workloads with shared system prompts at high concurrency.
- KV cache utilization will remain low (below 20%) for a 7B model on a 24 GB GPU even at high concurrency, making `gpu-memory-utilization` a non-bottleneck parameter.
- An FP8 KV cache will unlock substantially higher concurrency than the default cache by halving the per-token KV footprint, with the largest gains appearing at the highest tested concurrency.
- Aggressive KV-cache compression (TurboQuant) will save little VRAM on an 8B model because the cache is a small fraction of total VRAM, and its dequantization overhead will slow per-token decode.

- A hybrid Planner–Executor agent (commercial Planner, self-hosted Executor) will retain most of the accuracy of an all-commercial agent while cutting cost by roughly an order of magnitude.

Chapter 5

REQUIREMENTS SPECIFICATION

5.1 Functional Requirements

- (i) The system shall deploy vLLM with configurable parameters on both GKE (Kubernetes) and Lightning AI (bare-metal) environments.
- (ii) The experiment runner shall accept a JSON experiment definition file and execute each experiment sequentially, patching vLLM parameters between experiments without manual intervention.
- (iii) The load generator shall send HTTP requests to the vLLM OpenAI-compatible API endpoint using streaming SSE, and shall record TTFT, TPOT, ITL (P50/P99), E2E latency, output token count, and finish reason for each request.
- (iv) The GPU monitor shall sample `nvidia-smi` and the vLLM `/metrics` endpoint at configurable intervals and persist results to CSV.
- (v) The analysis notebook shall produce, for each experiment: latency distribution plots (violin, box), throughput bar charts, cost efficiency curves, KV cache time series, GPU utilization plots, and a Pareto frontier chart.
- (vi) The quality evaluator shall run `lm-evaluation-harness` against the vLLM API endpoint and store per-task scores.
- (vii) The budget alert system shall automatically disable GCP billing when cumulative spend exceeds the configured threshold.

5.2 Non-Functional Requirements

- (i) **Reproducibility:** Every experiment run shall be uniquely identified (timestamp-based run ID), and all configuration, results, and logs shall be stored together under a single run directory.

- (ii) **Scalability:** The pipeline shall support any number of experiments and concurrency levels defined in the JSON file without code changes.
- (iii) **Reliability:** The experiment runner shall handle vLLM restart failures gracefully (skipping the experiment and logging the failure) without aborting the entire campaign.
- (iv) **Observability:** All intermediate state (Locust stats, GPU metrics, vLLM metrics, Prometheus metrics) shall be persisted so that experiments can be re-analyzed without re-running.
- (v) **Cost safety:** Cloud GPU costs shall be bounded by the automated billing disable mechanism.

5.3 Use Cases

UC1: Run Experiment Campaign The researcher defines experiments in a JSON file and invokes the experiment runner. The runner iterates through all experiments, restarts vLLM as needed, executes Locust tests, collects metrics, and stores results.

UC2: Analyze Results The researcher opens the Jupyter analysis notebook, points it at the run directory, and executes all cells to produce latency/throughput/cost/GPU plots and summary tables.

UC3: Evaluate Model Quality The researcher invokes the quality evaluation script, which discovers the vLLM endpoint via `kubect1`, health-checks it, runs `lm-evaluation-harness` and stores scores.

UC4: Monitor Live Test During a running Locust test, the researcher monitors real-time metrics via the Grafana dashboard (GKE) or direct vLLM `/metrics` polling (Lightning AI).

Chapter 6

DESIGN

6.1 System Architecture

The system is organized into six major subsystems: Infrastructure, Load Generator, Experiment Runner, Telemetry Collection, Analysis Pipeline, and (for Phase 2) the Agent Application.

6.1.1 Infrastructure

On GKE, vLLM is deployed as a Kubernetes `Deployment` in the `vllm` namespace with one replica, one NVIDIA L4 GPU, 6 vCPUs, and 24 GiB memory. A `LoadBalancer Service` exposes the vLLM OpenAI API on port 80 (forwarded to port 8000 on the pod). Model configuration is stored in a `ConfigMap` (YAML file), and the HuggingFace token is stored in a `Secret`. Health probes (startup, liveness, readiness) guard against deploying a partially-initialized model.

On Lightning AI, vLLM runs as a direct `nohup` process managed by `serve.sh`. This eliminates Kubernetes rollout overhead and allows faster experiment iteration.

Monitoring on GKE uses the `kube-prometheus-stack` Helm chart (Prometheus + Grafana). A `ServiceMonitor` scrapes the vLLM `/metrics` endpoint; a `PodMonitor` scrapes DCGM GPU metrics from the GKE-managed DCGM exporter. A custom Locust dashboard (JSON) and official vLLM dashboards are provisioned into Grafana via a `ConfigMap`.

6.1.2 Load Generator (Locust)

The Locust load generator (`locustfile.py`) implements a single `VllmUser` task class that:

- (i) Randomly samples a prompt from the loaded dataset (ShareGPT or custom).

- (ii) Sends a streaming POST to `/v1/chat/completions` with `stream=True`.
- (iii) Parses the SSE stream token by token, recording the timestamp of each token.
- (iv) Computes TTFT, TPOT, ITL (P50/P99), E2E, output token count, and finish reason.
- (v) Appends the result to an in-memory list, which is flushed to CSV on test stop.

The load generator supports two dataset modes: ShareGPT (multi-turn conversations, auto-downloaded) and a custom dataset (`dataset.json`) with 37 prompts spanning coding, creative, reasoning, and general categories across short/medium/long lengths. A separate `prefix_dataset.json` provides shared-system-prompt prompts for prefix caching evaluation.

On GKE, Locust runs in a distributed master/worker configuration (1 master, 2 workers) using the official Locust Docker image with a Prometheus exporter on port 9646.

6.1.3 Experiment Runner

The experiment runner (`run_experiment.sh`) implements the following loop for each experiment:

- (i) Parse the JSON experiment definition (name, vLLM extra args, concurrency levels, prompt categories).
- (ii) Patch the vLLM deployment (GKE: `kubectl patch`; Lightning AI: `serve.sh restart`) and wait for the health endpoint to return 200.
- (iii) For each (concurrency, prompt category) combination: start the GPU/vLLM monitor, trigger the Locust test via the REST API, wait for the run duration, stop the monitor, and collect all output files.
- (iv) Save results under `results/<run_id>/<experiment_name>/`.

6.1.4 Telemetry Collection

GPU metrics are collected by `gpu_monitor.sh`, which polls `nvidia-smi` and the vLLM `/metrics` endpoint at 2-second intervals, writing GPU utilization, VRAM, power draw, temperature, KV cache percentage, running/waiting request counts, and generation throughput to separate CSV files.

On GKE, Prometheus metrics for the full test window are fetched post-hoc via the `fetch_metrics.sh` script, which queries the Prometheus `query_range` API for 27 pre-defined metrics derived from the Grafana dashboard panels.

6.1.5 Analysis Pipeline

The Jupyter notebook (`analyze_results.ipynb`) loads all CSVs from both experiment runs, computes summary statistics, and produces the following outputs:

- Violin and box plots for TTFT, TPOT, and E2E distributions by experiment and concurrency level.
- Heatmaps (experiment $\times$ concurrency) for P50/P99 TTFT, TPOT, and E2E.
- Pareto frontier scatter plot: output token throughput vs. P95 TTFT.
- Cost efficiency line charts: USD per million output tokens vs. concurrency.
- GPU utilization time series and KV cache time series per (experiment, concurrency).
- Prefix caching improvement bar chart and percentage change table.
- Quality bar charts and heatmaps from lm-evaluation-harness scores.
- Performance vs. quality scatter plots.

6.1.6 Agent Application (Phase 2)

The Phase 2 agent is a ReAct-style graph built with LangGraph and exposed through a FastAPI service. Its design follows the asymmetric “big brain, small hands” principle: an expensive, high-capability *Planner* produces a structured plan, and a cheap, self-hosted *Executor* carries it out using tools.

- **Planner:** By default a commercial model (Gemini 3 Flash via Vertex AI). Given a user query it identifies constraints (required output format, rounding, units), decomposes the task into sequential steps, and specifies a verification strategy. It is called sparingly—ideally once per query.
- **Executor:** By default the self-hosted Gemma 4 26B (AWQ 4-bit) served by the Phase 1 vLLM deployment through its OpenAI-compatible API. It consumes the plan and executes it step by step, calling tools and adapting when a step fails. It handles the overwhelming majority of token volume, where the near-zero marginal cost of self-hosting dominates.
- **Tool suite:** `web_search` (DuckDuckGo), `visit_webpage` (HTML-to-markdown scraping), `read_document` (multi-format file parsing), and `python_execute` (sandboxed code execution in an E2B cloud sandbox). A “mock” tool mode returns static fixtures for load-testing the serving path without external dependencies, while a “real” mode performs live calls for GAIA evaluation.

- **State and observability:** An optional Postgres checkpointer (`AsyncPostgresSaver`) persists conversation state, and every planner/executor/tool span is traced to Langfuse for token accounting and debugging.

Five modes are selectable at deploy time via a single `MODE` setting, forming the experimental matrix for Phase 2: `large-planner-executor` (commercial Planner + commercial Executor, the quality ceiling), `planner-executor` (commercial Planner + self-hosted Executor, the proposed hybrid), `planner-only` (commercial model used directly), `single-agent` (self-hosted model used directly), and `small-planner-executor` (self-hosted Planner + self-hosted Executor, the fully-local floor).

6.2 Information Structure

Each benchmark run produces the following data artifacts, organized in a directory tree:

```
results/
  run_<YYYYMMDD_HHMMSS>/
    <experiment_name>/
      config.json           # experiment parameters
      <label>_custom_metrics.csv # per-request Locust metrics
      <label>_stats.csv       # Locust aggregate stats
      <label>_gpu_metrics.csv  # nvidia-smi time series
      <label>_vllm_metrics.csv # vLLM /metrics time series
      <label>_prometheus_metrics.csv # Prometheus time series
      <label>_prompt_responses.jsonl # prompts + model responses
```

The custom metrics CSV has one row per request, with columns: `timestamp`, `category`, `approx_chars`, `output_tokens`, `ttft_ms`, `tpot_ms`, `e2e_ms`, `itl_p50_ms`, `itl_p99_ms`, `finish_reason`, `success`, `error_type`.

6.3 Information Flow

The high-level information flow is:

- (i) The experiment runner reads the JSON experiment file and drives the outer control loop.
- (ii) For each experiment, it configures the vLLM server and signals Locust to start a swarm.

- (iii) Locust workers generate HTTP requests to vLLM, parse SSE responses, and accumulate per-request metrics in memory.
- (iv) In parallel, the GPU monitor polls `nvidia-smi` and `/metrics` and writes CSVs.
- (v) On test stop, Locust workers flush their metric buffers to CSV; the experiment runner copies these files to the results directory.
- (vi) After all experiments, the Jupyter notebook reads the results directory and produces the analysis outputs.
- (vii) The quality evaluator sends requests to the same vLLM endpoint and records lm-eval scores.

6.4 Deployment Architecture

6.4.1 GKE Deployment

The GKE cluster is provisioned in `europa-west3-b` (Frankfurt) with a CPU node pool (`n2d-standard-4`) and a GPU node pool (`g2-standard-8` with 1 NVIDIA L4). Node affinity rules ensure Locust pods are scheduled on CPU nodes and the vLLM pod on the GPU node. NVIDIA GPU drivers are installed via the Google-managed DCGM daemonset.

6.4.2 Lightning AI Deployment

For experiments run on Lightning AI, vLLM is started directly via `vllm serve` with parameters read from `config.env`. The experiment runner calls `serve.sh restart` between experiments, passing the new extra args. This avoids Kubernetes rollout latency (approximately 15 minutes per restart on GKE) and reduces iteration time significantly.

6.5 User Interface Design

The primary user interfaces are:

- (i) **Grafana Dashboard:** Real-time visualization of TTFT/TPOT/E2E/ITL percentiles, KV cache utilization, GPU metrics, and Locust request rates during live tests. The dashboard JSON is provisioned automatically by the monitoring deployment script.

- (ii) **Locust Web UI:** Available at `localhost:8089` via port-forward, showing live request statistics and charts during a test run.
- (iii) **Jupyter Notebook:** Post-experiment analysis interface with fully reproducible cells.
- (iv) **CLI:** All experiment management is shell-script-based for easy integration into CI/CD pipelines.

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

7.1.1 vLLM Deployment Scripts

The vLLM deployment is managed by three shell scripts: `gke.sh` (cluster and GPU node pool creation), `deploy.sh` (Kubernetes manifest application, ConfigMap creation, rollout wait), and `serve.sh` (Lightning AI start/stop/restart). Configuration is separated into `infra_config.env` (cluster parameters), `vllm_config.yaml` (model serving parameters), and `secrets.env` (HuggingFace token).

The vLLM Kubernetes deployment uses a `Recreate` rollout strategy (rather than `RollingUpdate`) because the GPU node has only one GPU; a rolling update would stall indefinitely waiting for a second GPU slot that does not exist. Startup, liveness, and readiness probes check the `/health` endpoint with generous timeouts to accommodate model weight download and loading times (up to 20 minutes on first run).

7.1.2 Locust Load Generator

The Locust implementation extends the base `HttpUser` class with `wait_time = constant(0)` (no think time between requests) so that the concurrency level alone controls load intensity. The SSE streaming parser uses the `sseclient-py` library to iterate over Server-Sent Events and extracts token-level timing.

Special handling is implemented for reasoning models: the `choices[0].delta.reasoning_content` field (used by vLLM for DeepSeek-R1’s chain-of-thought) is accumulated separately from `choices[0].delta.content`. A fallback regex extracts `<think>...</think>` blocks from models that embed reasoning inline in the content stream.

A Prometheus exporter (port 9646) exposes `locust_users`, `locust_fail_ratio`,

and `locust_requests_total` for real-time monitoring.

7.1.3 Experiment Runner

The GKE experiment runner (`benchmarking/run_experiment.sh`) implements the full automation loop including: `kubectl patch` to update vLLM container args, roll-out status waiting (up to 1200s), `kubectl port-forward` to the Locust master (for API control), Prometheus port-forward (for post-test metric fetch), Locust swarm start/stop via the master REST API, per-worker `kubectl cp` to retrieve custom metrics CSVs and response JSONL files, and Prometheus `query_range` fetch via `fetch_metrics.sh`.

The Lightning AI experiment runner (`lightning-setup/run_experiment.sh`) replaces Kubernetes operations with direct `serve.sh` calls and local GPU monitoring, reducing overhead substantially.

7.1.4 Budget Guard

A Cloud Function (`budget/function/main.py`) is triggered via Pub/Sub when GCP billing exceeds the configured threshold. It calls the Cloud Billing API to disable billing for the project, preventing runaway costs. This is particularly important for GPU workloads where an unattended overnight run can accumulate significant charges.

7.1.5 Quality Evaluation Pipeline

The quality evaluator (`evaluation/run_eval.sh`) discovers the vLLM service endpoint via `kubectl`, health-checks it, and invokes `lm_eval` in `local-completions` mode against the vLLM API. For reasoning models, `enable_thinking=true` is passed to ensure the chain-of-thought is generated but the answer is extracted from the content field. `max_gen_toks=4096` is set to prevent truncated answers for long reasoning chains.

7.1.6 Phase 2 Agent Application

The agent is implemented as a FastAPI service (`agent/app/main.py`) wrapping a LangGraph graph (`agent/app/graph.py`). Two factory functions, `_create_planner_llm` and `_create_executor_llm`, instantiate the right model for the active `MODE`: a `ChatVertexAI` client for the commercial Gemini Planner/Executor, or a `ChatOpenAI` client pointed at the in-cluster vLLM service for the self-hosted side. The planner is prompted to emit a structured plan (constraints, ordered steps, and a verification strategy) tailored to GAIA-style tasks, and the executor is prompted with strict rules—most importantly a “Python-first” rule that forbids doing arithmetic or data parsing in-context and instead routes it to the `python_execute` tool.

Tools are provided in two interchangeable suites. The *mock* suite (`mock_tools.py`) returns static fixtures served by a separate `mock-tools` microservice, used to load-test the serving path deterministically. The *real* suite (`real_tools.py`) performs live work: `web_search` queries DuckDuckGo with retry/backoff, `visit_webpage` fetches and converts pages to markdown (truncated to 20,000 characters), `read_document` parses PDF/DOCX/CSV/Excel files, and `python_execute` runs model-generated code inside an ephemeral E2B sandbox, uploading any per-session files first. State is persisted with an `AsyncPostgresSaver` checkpointer, and all spans are exported to Langfuse for token-level cost accounting.

7.1.7 GAIA Evaluation Harness

The GAIA runner (`evaluation/gaia/run_gaia.py`) loads the gated `gaia-benchmark/GAIA` validation split, filters to the requested levels, and dispatches each question to the agent API with a bounded concurrency (default 4) and a per-question timeout. Answers are graded by an exact-match scorer (`scorer.py`) that lowercases, strips whitespace, normalizes numbers, and removes currency/unit symbols before comparison. Each run writes a per-question `results.jsonl` and an aggregate `summary.json` with overall and per-level accuracy. Token-usage and cost figures for each configuration are reconciled from Langfuse traces and summarized into the `comparison.csv` and `comparison_cost_adjusted.csv` tables used in the analysis.

7.2 Testing

7.2.1 Unit-Level Verification

Each component was verified independently:

- **SSE parser:** Verified against recorded vLLM SSE responses to confirm correct token counting and TTFT computation.
- **Experiment runner dry-run:** The `-dry-run` flag prints the full experiment plan without touching the cluster, used to verify the experiment matrix before each campaign.
- **GPU monitor:** Verified by comparing `_gpu_metrics.csv` time series against concurrent Grafana readings.
- **vLLM health check:** The `ping.sh` script sends a single chat completion request and measures E2E latency, used as a quick sanity check after each deployment.

7.2.2 Integration Testing

End-to-end pipeline testing was performed by running a single-experiment, single-concurrency campaign and manually verifying that all expected output files were produced, that metric values fell within physically plausible ranges (e.g., TTFT > 0, TPOT > 0, GPU utilization 0–100%), and that the analysis notebook produced plots without errors.

7.2.3 Experiment Validation

The 100% success rate across 2,041 requests (all experiments combined) provides strong evidence that the pipeline is reliable and that vLLM did not crash or return errors under any tested configuration.

7.3 Deployment

7.3.1 GKE Deployment Procedure

- (i) `bash prerequisites.sh` — verify tools and enable GCP APIs.
- (ii) `bash gke.sh` — create cluster and GPU node pool.
- (iii) `bash monitoring/deploy.sh` — deploy Prometheus, Grafana, and dashboards.
- (iv) `bash locust/deploy.sh` — build and deploy Locust master/workers.
- (v) `bash vllm/scripts/deploy.sh` — deploy vLLM.
- (vi) `bash benchmarking/run_experiment.sh -experiment-file experiments/experiments.json`

7.3.2 Lightning AI Deployment Procedure

- (i) `bash setup.sh` — install vLLM, Locust, and dependencies.
- (ii) `export HF_TOKEN=hf_XXXXX`
- (iii) `bash run_experiment.sh -experiments ../benchmarking/experiments.json`

Chapter 8

RESULTS

This chapter presents the results of all benchmark campaigns. It is organized in two parts. **Part I (Phase 1)** covers the serving-optimization experiments: the pilot server-parameter sweep, weight quantization on the DeepSeek-R1 and Qwen2.5 7B models, I/O-profile analysis, and the extended studies on Qwen3-8B (reasoning mode, FP8 KV cache, and TurboQuant) and the Gemma 4 vs. Qwen3 comparison. **Part II (Phase 2)** covers the GAIA Level 1 evaluation of the Planner–Executor agent. All serving experiments achieved at least a 99.7% request success rate; the Phase 1 campaigns together processed well over 5,000 successfully completed requests (2,041 in the initial 7B weight-quantization campaign, 2,274 in the Qwen3-8B campaigns, and 1,093 in the TurboQuant campaign).

Part I — Phase 1: Serving Optimization

8.1 Pilot Study: vLLM Server Parameter Sweep

An initial parameter sweep was conducted on Lightning AI using the custom 37-prompt dataset. Eight configurations were tested at 1, 4, 8, 16, and 32 concurrent users.

8.1.1 Baseline Performance

At baseline (chunked prefill OFF, prefix caching OFF, `max-num-seqs` = 128 (default), `gpu-memory-utilization` = 0.90), the following P50 latencies were observed:

The E2E latency is approximately $256 \times 56 \text{ ms} + 75 \text{ ms} \approx 14,411 \text{ ms}$, matching the theoretical expectation almost exactly (within 0.4%). TPOT remains stable across concurrency levels because decode throughput is memory-bandwidth-bound and is nearly independent of batch size in this range.

Table 8.1: Baseline latency at different concurrency levels (DeepSeek-R1, FP16, Lightning AI L4).

Users	P50 TTFT (ms)	P50 TPOT (ms/tok)	P50 E2E (ms)
1	75	56	14,464
4	147	58	14,802
8	178	58	14,933

8.1.2 Effect of max-num-seqs

Reducing `max-num-seqs` to 8 showed negligible performance difference from baseline at low concurrency. Increasing to 32 and testing at 32 users achieved the highest throughput observed in the pilot (455 tok/s) but at the cost of higher TTFT (276 ms P50 at u=32 vs. 178 ms at u=8 for baseline).

8.1.3 Effect of gpu-memory-utilization

Changing `gpu-memory-utilization` between 0.80 and 0.95 showed no measurable performance difference. KV cache utilization peaked at only 5.4% across all configurations, confirming that a 7B model on a 24 GB GPU does not exhaust the KV cache at the tested concurrency levels.

8.1.4 Effect of Chunked Prefill

With the short prompts used in the pilot dataset (maximum 127 tokens), chunked prefill showed no measurable improvement. This is expected: chunked prefill’s primary benefit is reducing head-of-line blocking when long prompts monopolize prefill compute, which does not occur with short prompts.

8.1.5 Effect of Prefix Caching

A dedicated Run 2 using shared-system-prompt prompts demonstrated the following TTFT improvements relative to the no-caching baseline:

Table 8.2: Prefix caching TTFT P50 improvement vs. baseline (shared-prefix dataset).

Configuration	u=1	u=4	u=8
<code>prefix_caching_on</code>	−9.2%	−35.4%	−53.7%
<code>prefix_chunked_plus_cache</code>	−1.8%	−39.8%	−55.7%

The improvement grows with concurrency because later requests find pre-warmed cache entries. At u=8, TTFT drops by over half. This is a major benefit for production chatbot applications where the same system prompt is sent with every request.

8.1.6 Throughput and Cost in the Pilot

Output token throughput scaled approximately linearly with concurrency: from approximately 90 tok/s at $u=1$ to 455 tok/s at $u=32$ with `max-num-seqs=32`. The corresponding cost per million output tokens dropped from approximately \$21 at $u=1$ to approximately \$0.90 at $u=32$, a $23\times$ reduction. This illustrates the fundamental economics of LLM serving: GPU time is fixed, so throughput directly determines cost efficiency.

8.2 Main Campaign: Quantization — Reasoning Model (DeepSeek-R1)

8.2.1 FP8 Quantization

FP8 quantization delivers substantial latency improvements over the FP16 baseline:

Table 8.3: DeepSeek-R1 quantization comparison at 16 concurrent users.

Configuration	TTFT P50 (ms)	TPOT P50 (ms/tok)	E2E P50 (ms)	Throughput (tok/s)
Baseline (FP16)	468	69	71,382	262
FP8	338	45	46,503	301
BitsAndBytes	689	138	200,770	136

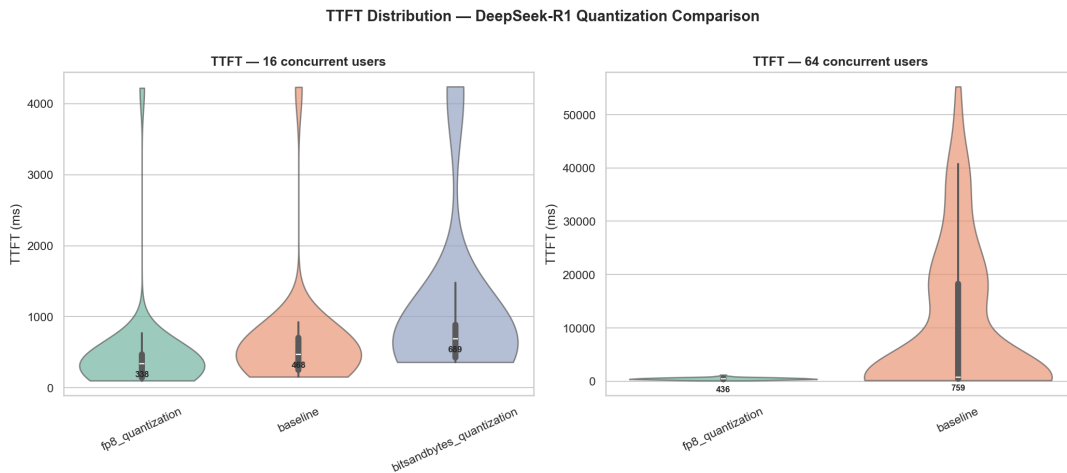


Figure 8.1: TTFT distribution by quantization method for the reasoning model (DeepSeek-R1). FP8 shifts the distribution lower; BitsAndBytes shifts it higher.

FP8 reduces TTFT P50 by 28%, TPOT P50 by 35%, and E2E P50 by 35% compared to the FP16 baseline. At 64 concurrent users, the TPOT improvement is $111 \rightarrow 76$ ms/tok (-31%). These are large gains for a single configuration change.

8.2.2 BitsAndBytes (INT4) Quantization

BitsAndBytes performs severely worse than both baseline and FP8:

- TTFT P50: 689 ms vs. 468 ms baseline (+47%)
- TPOT P50: 138 ms/tok vs. 69 ms/tok (+100%)
- E2E P50: 200,770 ms vs. 71,382 ms (+181%)
- Cost: \$3.22/M tokens vs. \$1.67/M for baseline (+93% more expensive)

BitsAndBytes nearly doubles the per-token generation time. This occurs because the L4 GPU does not have native INT4 computation support; BitsAndBytes performs dequantization to FP16 at runtime, adding overhead rather than reducing it.

8.3 Main Campaign: Quantization — Non-Reasoning Model (Qwen2.5-7B-Instruct)

FP8 quantization again improves TTFT significantly for the non-reasoning model:

Table 8.4: Qwen2.5 quantization comparison at 16 concurrent users.

Configuration	TTFT P50 (ms)	TPOT P50 (ms/tok)	E2E P50 (ms)
Baseline (FP16)	574	72	41,955
FP8	403	70	39,369

The TTFT improvement (−30%) is comparable to the reasoning model. TPOT improvement is modest (−3%) because the non-reasoning model generates fewer output tokens, reducing the proportion of inference time spent in the decode phase where FP8’s compute benefits are largest.

8.4 Reasoning vs. Non-Reasoning Model Comparison

Table 8.5: Reasoning vs. non-reasoning model at FP16 baseline, 16 concurrent users.

Model	TTFT P50 (ms)	TPOT P50 (ms/tok)	E2E P50 (ms)
DeepSeek-R1 (reasoning)	468	69	71,382
Qwen2.5 (non-reasoning)	574	72	41,955

Despite similar per-token generation speeds (TPOT is nearly identical), the reasoning model’s E2E latency is 70% higher because it generates a chain-of-thought block before the answer, producing far more output tokens per request. For applications where only the final answer is needed, the non-reasoning model offers much

faster response times. For reasoning-intensive tasks (mathematics, code generation with complex logic), the reasoning model’s quality advantage must be weighed against this latency cost.

8.5 Main Campaign: I/O Profile Analysis

Six prompt profiles were tested to understand how input/output characteristics affect latency. Results are for DeepSeek-R1 with FP8 quantization.

8.5.1 TTFT by Profile

TTFT remains remarkably consistent across all profiles (≈ 190 ms at $u=16$, ≈ 260 ms at $u=64$), regardless of whether prompts are short (coding, general), medium (reasoning, creative), or long (detailed system design questions). This confirms that within the 4096-token context window, the prefill phase is not a bottleneck on the L4 GPU for these prompt lengths.

8.5.2 E2E Latency by Profile

E2E latency varies substantially with profile, driven entirely by output token count:

Table 8.6: E2E P50 latency by I/O profile (FP8, DeepSeek-R1).

Profile	E2E P50 at $u=16$ (ms)	E2E P50 at $u=64$ (ms)
short_input	36,964	52,883
all (mixed)	92,864	117,772
creative	101,756	151,591
coding	124,261	149,412
reasoning	124,599	164,183
long_input	126,043	169,961

Short-input prompts finish $3.4\times$ faster than long-input prompts at $u=16$. This is driven by output length, not input length: with `max_tokens=2048`, the reasoning model tends to generate longer chains of thought for complex inputs.

8.6 Throughput and Cost Analysis

The lowest cost configuration is DeepSeek-R1 FP8 at $u=64$: \$0.47/M output tokens. This is $6.85\times$ cheaper than BitsAndBytes at $u=16$ (\$3.22/M). Higher concurrency (more requests batched together per GPU forward pass) is the single biggest driver of cost reduction, followed by FP8 quantization.

Table 8.7: Cost per million output tokens at different configurations and concurrency levels.

Configuration	u=16 (\$/M tok)	u=64 (\$/M tok)
DeepSeek-R1 FP16 baseline	\$1.67	\$0.68
DeepSeek-R1 FP8	\$1.46	\$0.47
DeepSeek-R1 BitsAndBytes	\$3.22	—
Qwen2.5 FP16 baseline	\$2.06	\$0.81
Qwen2.5 FP8	\$1.91	—

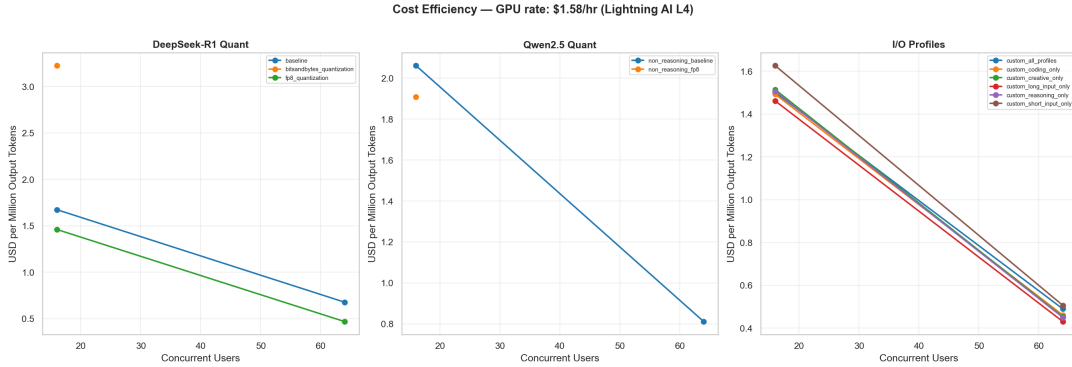


Figure 8.2: Cost per million output tokens vs. concurrency across configurations. Cost falls steeply with concurrency; FP8 is consistently cheaper than the FP16 baseline.

8.7 Quality Evaluation

Table 8.8: lm-evaluation-harness quality scores by model and quantization.

Model	Quantization	ARC-Challenge	MMLU	GSM8K
DeepSeek-R1	FP16 (baseline)	0.526	0.593	0.800
DeepSeek-R1	FP8	0.518	0.579	0.785
DeepSeek-R1	BitsAndBytes	0.516	0.593	—
Qwen2.5	FP16 (baseline)	0.662	0.782	0.881
Qwen2.5	FP8	0.670	0.779	0.879
Qwen2.5	BitsAndBytes	0.644	—	0.861

FP8 quantization causes negligible quality degradation: the maximum reduction is -2.37% (MMLU for DeepSeek-R1). Qwen2.5 FP8 even shows a slight gain on ARC-Challenge ($+1.21\%$), which is within measurement noise. BitsAndBytes causes up to -2.72% degradation (ARC-Challenge for Qwen2.5).

Qwen2.5 outperforms DeepSeek-R1 on all three benchmarks, reflecting its larger training dataset and more recent training methodology.

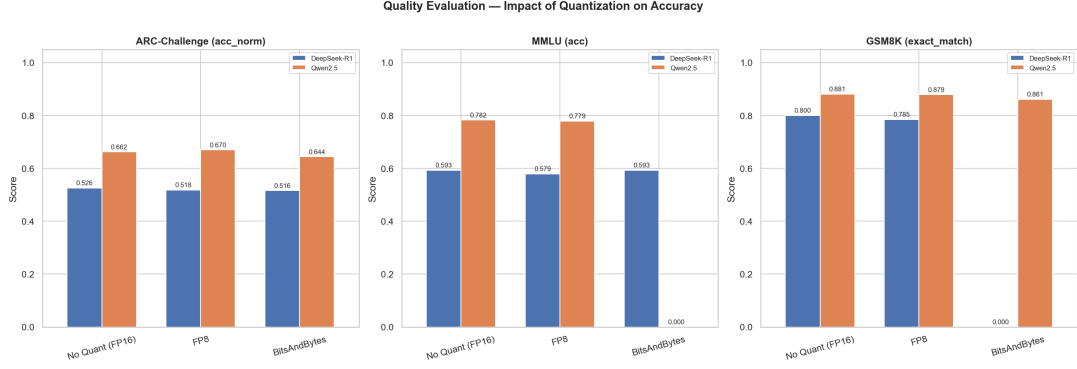


Figure 8.3: lm-evaluation-harness scores (ARC-Challenge, MMLU, GSM8K) by model and quantization. FP8 tracks the FP16 baseline closely across all benchmarks.

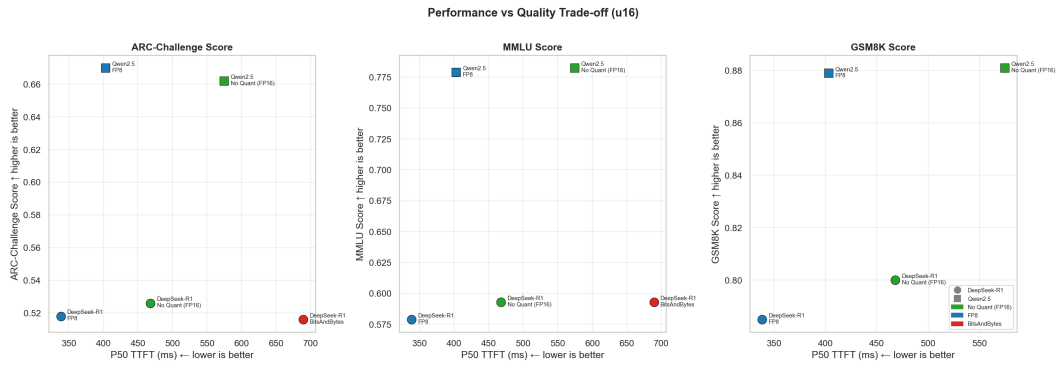


Figure 8.4: Performance–quality trade-off. FP8 sits on the favourable frontier: near-baseline quality at materially lower latency and cost.

8.8 Data Completeness

Some experiments have partial data due to infrastructure issues encountered during the campaign:

Table 8.9: Data completeness across experiment campaigns.

Campaign	Experiment	Custom Metrics	Prometheus	Locust Stats
Quant Reasoning	baseline (u64)	✓	✓	✓
Quant Reasoning	fp8 (u64)	✓	✓	✓
Quant Reasoning	bitsandbytes (u16)	×	×	✓
Quant Non-Reasoning	non_reasoning_baseline (u64)	✓	✓	✓
I/O Profiles	all (u16 and u64)	✓	✓	✓

Full per-request custom metrics (TTFT, TPOT, ITL) were available for the I/O profiles campaign and for key quantization configurations. Some early quantization runs captured only Locust aggregate statistics, which still provide sufficient data for the primary latency and throughput comparisons.

8.9 Extended Study: Qwen3-8B Reasoning vs. Non-Reasoning

After the initial 7B campaigns, the study moved to Qwen3-8B, a dual-mode model that exposes reasoning via a single `enable_thinking` switch. This allows a clean within-model comparison of the two modes under identical load (run `run_20260404_181323`, 16/32/64 concurrent users).

Table 8.10: Qwen3-8B reasoning vs. non-reasoning, P50 latencies.

Metric	Reasoning	Non-Reasoning	Change
TTFT P50 (u16)	460 ms	421 ms	−8%
TPOT P50 (u16)	62.1 ms/tok	60.2 ms/tok	−3%
E2E P50 (u16)	92,313 ms	49,705 ms	−46%
E2E P50 (u32)	127,119 ms	58,777 ms	−54%
E2E P50 (u64)	166,331 ms	94,475 ms	−43%

Non-reasoning mode produces 43–54% lower E2E latency at every concurrency level because it omits the chain-of-thought tokens. TTFT and TPOT are comparable at low concurrency, but at u64 the non-reasoning configuration’s TTFT degrades sharply due to queue buildup as more (shorter) requests compete for KV cache. Throughput

tells the same story: non-reasoning scales to 575 tok/s at u64 (\$0.76/M tokens) while reasoning drops to 320 tok/s as longer sequences trigger preemptions. Critically, quality is essentially identical between modes (ARC 0.656/0.656, MMLU 0.786/0.786, GSM8K 0.899/0.896), so for latency-sensitive deployments the non-reasoning mode is the better operating point on these benchmarks.

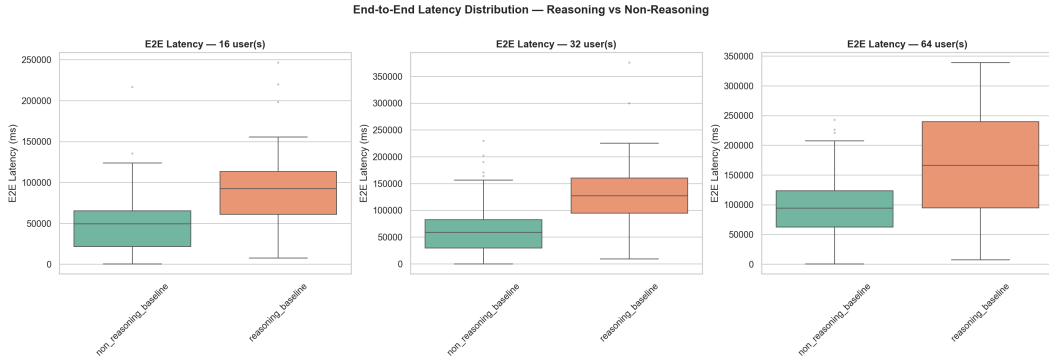


Figure 8.5: End-to-end latency distribution, Qwen3-8B reasoning vs. non-reasoning. The reasoning mode’s chain-of-thought roughly doubles E2E latency.

8.10 KV Cache Precision: Default vs. FP8 (Qwen3-8B)

A second Qwen3-8B campaign (run_20260405_005704) compared the default (16-bit) KV cache against an FP8 KV cache, scaling concurrency up to 128 users.

Table 8.11: Qwen3-8B KV cache: default (auto) vs. FP8, selected metrics.

Metric	Auto	FP8	Change
TTFT P50 (u64)	30,433 ms	405 ms	−99%
TTFT P50 (u128)	143,695 ms	36,216 ms	−75%
E2E P50 (u128)	188,400 ms	124,353 ms	−34%
Throughput (u128)	313 tok/s	901 tok/s	+188%
Cost (u128)	\$1.40/M	\$0.49/M	−65%
Waiting reqs (u128)	61.2	22.5	−63%

The FP8 KV cache is the single most impactful Phase 1 lever at high concurrency. By halving the per-token KV footprint, it admits far more concurrent sequences before queuing: at u64 the default cache’s TTFT collapses to 30 seconds while FP8 holds 405 ms, and at u128 FP8 sustains 901 tok/s—roughly 3× the default’s 313 tok/s—reaching the lowest cost of \$0.49/M tokens. Quality is unaffected (ARC 0.660, MMLU 0.797, GSM8K 0.879). This confirms the hypothesis that FP8 KV cache unlocks concurrency with negligible quality cost, and it became the default serving configuration for all subsequent work, including the Phase 2 Executor.

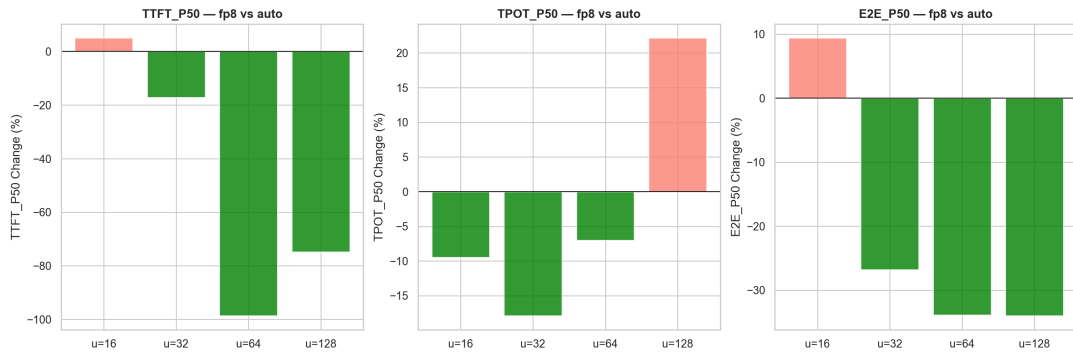


Figure 8.6: FP8 KV cache improvement over the default cache (Qwen3-8B). Gains are largest at the highest concurrency, where the default cache saturates and queues.

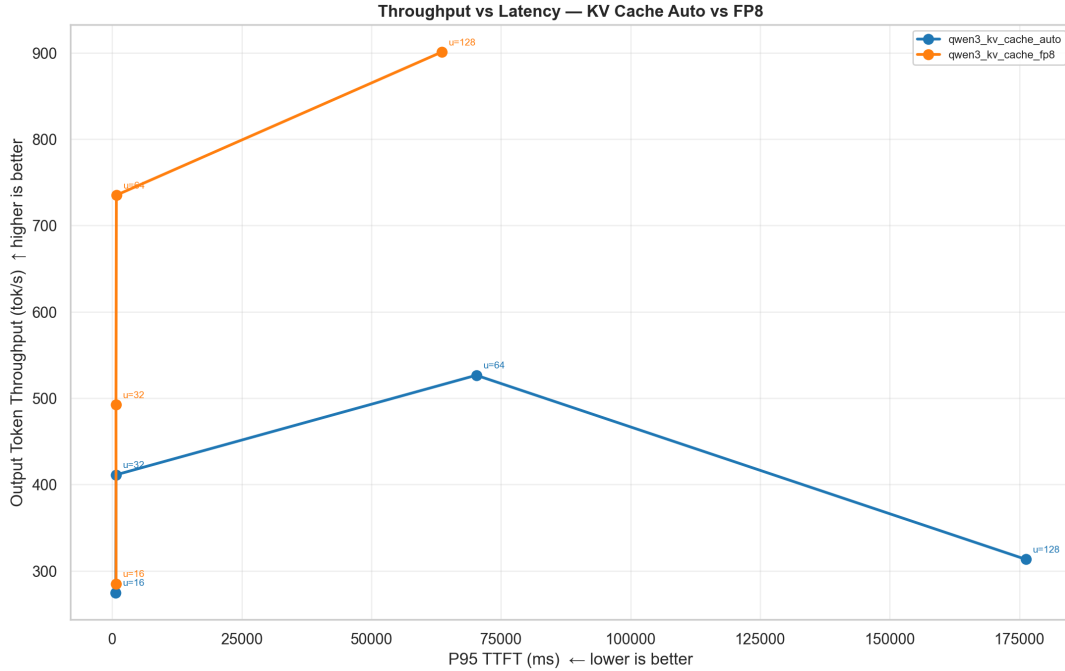


Figure 8.7: Throughput vs. latency Pareto frontier, default vs. FP8 KV cache. FP8 dominates the frontier across concurrency levels.

8.11 TurboQuant KV-Cache Compression

Having established FP8 as a strong baseline, the study tested whether more aggressive KV-cache compression (TurboQuant) could push further. Five presets were compared on Qwen3-8B (runs `run_20260511_234500` and `run_20260512_090311`) at 32/64/128 users: an FP8 baseline, `k8v4` (FP8 keys + 4-bit values), `4bit_nc` (4-bit keys+values with norm correction), `k3v4_nc` (3-bit keys + 4-bit values), and `3bit_nc` (3-bit keys + 3-bit values).

Table 8.12: TurboQuant vs. FP8 KV baseline (Qwen3-8B): throughput and cost.

Config	u128 tok/s	u128 \$/M	TPOT P50 vs. base
fp8 (baseline)	1,097	\$0.40	—
tq k8v4	292	\$1.50	+28%
tq 4bit_nc	575	\$0.76	+59%
tq k3v4_nc	760	\$0.58	+88%
tq 3bit_nc	392	\$1.12	+106%

The results are clear and somewhat counter-intuitive: on the L4 with an 8B model, TurboQuant is *not* worth it. Three findings dominate. First, the VRAM saving is tiny (~ 400 MiB out of 20.6 GiB) because the KV cache is only a small fraction of total VRAM—weights, activations, and allocator overhead dominate—so the compression does not free enough headroom to admit more concurrent requests (mean KV usage at u128 sits at 90–93% for every variant including the baseline). Second, the dequantization compute lands directly on the decode path, inflating TPOT by 28–106% and roughly doubling ITL on the 3-bit variants. Third, quality is essentially unchanged (± 2.5 pp on ARC and GSM8K), so the compression buys no quality headroom either. The one niche advantage is TTFT under saturation: at u128 the smaller KV blocks let new requests admit 9–28% faster, so a TurboQuant preset is defensible only when first-token responsiveness at extreme load matters more than throughput. Otherwise FP8 KV remains the best operating point at \$0.40/M tokens.

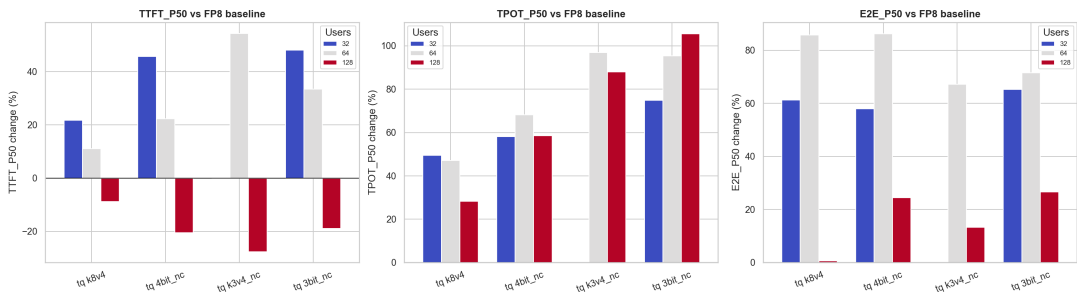


Figure 8.8: Percentage change of each TurboQuant preset relative to the FP8 KV baseline. Per-token decode (TPOT/ITL) regresses uniformly; throughput falls.

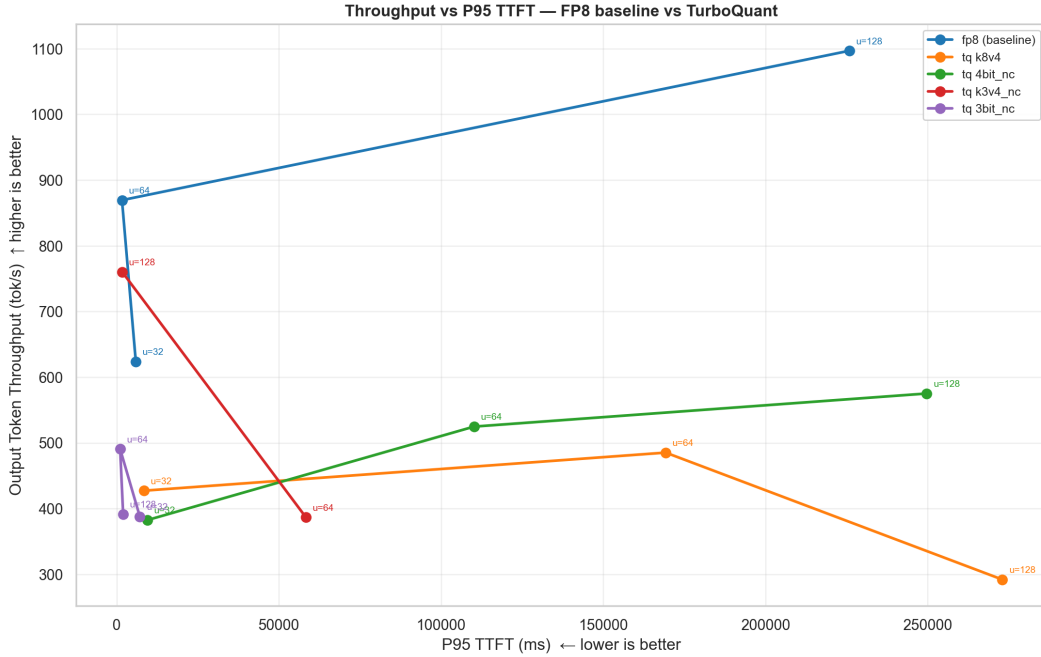


Figure 8.9: Throughput vs. P95 TTFT Pareto frontier for TurboQuant presets. The FP8 baseline dominates on throughput; TurboQuant only wins on admit-time TTFT under saturation.

8.12 Model Comparison: Gemma 4 vs. Qwen3-8B

Finally, the serving study compared two model families at matched FP8 KV configuration (run `run_20260416_010036` vs. the Qwen3 FP8 run): Google’s Gemma 4 E4B against Qwen3-8B.

Table 8.13: Gemma 4 E4B vs. Qwen3-8B (FP8 KV), selected metrics.

Metric	Gemma 4 E4B	Qwen3-8B
TTFT P50 (u128)	525 ms	36,216 ms
TPOT P50 (u128)	102.0 ms/tok	124.7 ms/tok
Throughput (u128)	1,880 tok/s	3,599 tok/s
Cost (u128)	\$0.23/M	\$0.12/M
ARC-Challenge	0.31	0.65
GSM8K	0.70	0.88

The two models occupy different niches. Gemma 4 E4B is extraordinarily resilient to queueing—at u128 it holds a 525 ms TTFT where Qwen3 spikes to an unusable 36.2 seconds—and has slightly better TPOT at every level. But Qwen3-8B delivers far higher peak throughput (3,599 vs. 1,880 tok/s, making it nearly twice as cost-effective at \$0.12/M) and decisively higher quality (ARC 65% vs. 31%, GSM8K 88% vs. 70%). For a throughput- and quality-oriented serving backend, Qwen3-8B is preferred; Gemma 4’s stability advantage is most valuable for latency-critical, interactive workloads.

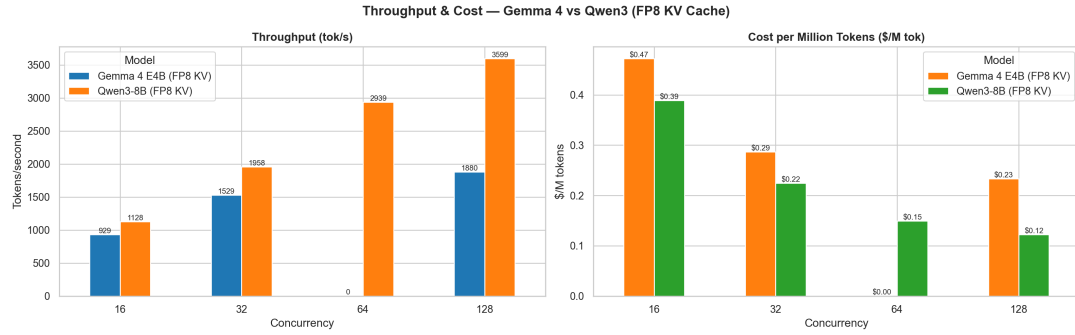


Figure 8.10: Throughput and cost, Gemma 4 E4B vs. Qwen3-8B (FP8 KV). Qwen3 reaches far higher peak throughput and lower cost per token.

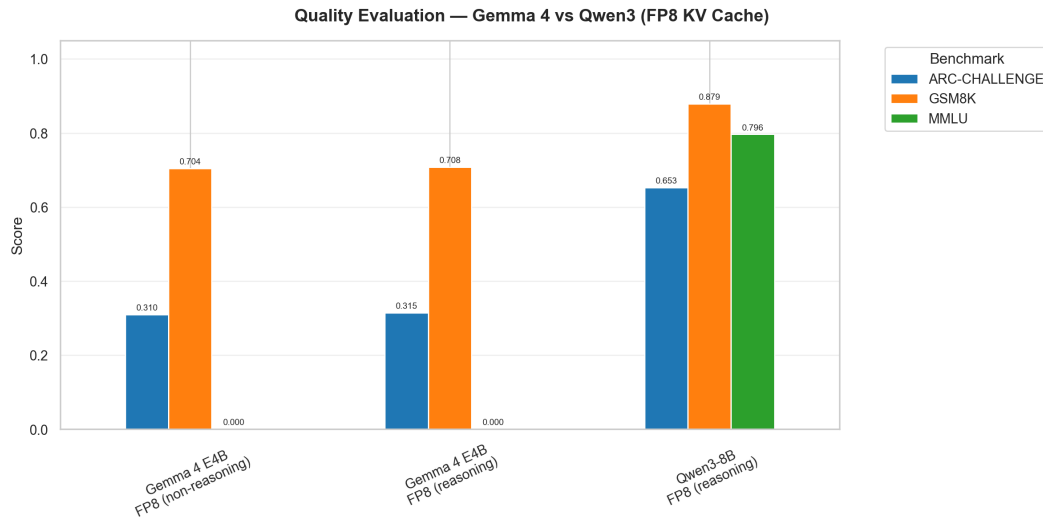


Figure 8.11: Quality comparison, Gemma 4 E4B vs. Qwen3-8B. Qwen3 substantially outscores Gemma 4 on ARC-Challenge and GSM8K.

Part II — Phase 2: Asymmetric Agent on GAIA

Phase 2 evaluates five agent configurations on the GAIA Level 1 validation set (53 multi-step questions requiring web search, file handling, and code execution). The Executor, where self-hosted, is the AWQ 4-bit Gemma 4 26B served by the FP8-optimized vLLM stack from Phase 1; the Planner, where commercial, is Gemini 3 Flash. Each configuration is measured on accuracy, mean task duration, and cost per question.

Table 8.14: GAIA Level 1 results across agent configurations. Cost is the cost-adjusted figure (self-hosted Executor tokens imputed a price).

Configuration	Accuracy	Cost/Q	Duration
Large-Planner-Executor (Gemini + Gemini)	84.9%	\$0.1207	356 s
Planner-Only (Gemini direct)	73.6%	\$0.0363	319 s
Planner-Executor (Gemini + Gemma 4)	67.9%	\$0.0110	278 s
Single-Agent (Gemma 4 direct)	50.9%	\$0.0050	106 s
Small-Planner-Executor (Gemma 4 + Gemma 4)	39.6%	\$0.0021	651 s

The headline result is the **hybrid Planner–Executor** configuration (commercial Gemini Planner + self-hosted Gemma 4 Executor). It achieves 67.9% accuracy—about 80% of the all-commercial ceiling of 84.9%—while costing \$0.0110 per question, roughly **one-tenth** of the all-commercial baseline’s \$0.1207. In other words, offloading the high-volume execution work to the self-hosted model preserves four-fifths of the quality at one-tenth of the cost, placing the hybrid on the Pareto frontier of the accuracy-versus-cost space.

The other configurations frame this result. The all-commercial **Large-Planner-Executor** sets the quality ceiling but is by far the most expensive. **Planner-Only**, calling Gemini directly without a separate executor, is cheaper but loses 11 pp of accuracy—confirming that an explicit plan-then-execute decomposition helps. At the low end, the fully self-hosted **Single-Agent** reaches a respectable 50.9% at near-zero marginal cost, while the fully self-hosted **Small-Planner-Executor** actually *regresses* to 39.6% and the longest duration (651 s): a weak planner produces verbose, sometimes misleading plans that the same weak model then struggles to execute, so the two-model overhead is not repaid. The clear lesson is that planning quality is what must be bought from the frontier model; execution can be safely and cheaply self-hosted.

To validate that the self-hosted Executor can sustain agentic traffic, the Phase 1 Locust harness was extended with an agent workload (`locustfile_agent.py`, `agent_load_test.json`)

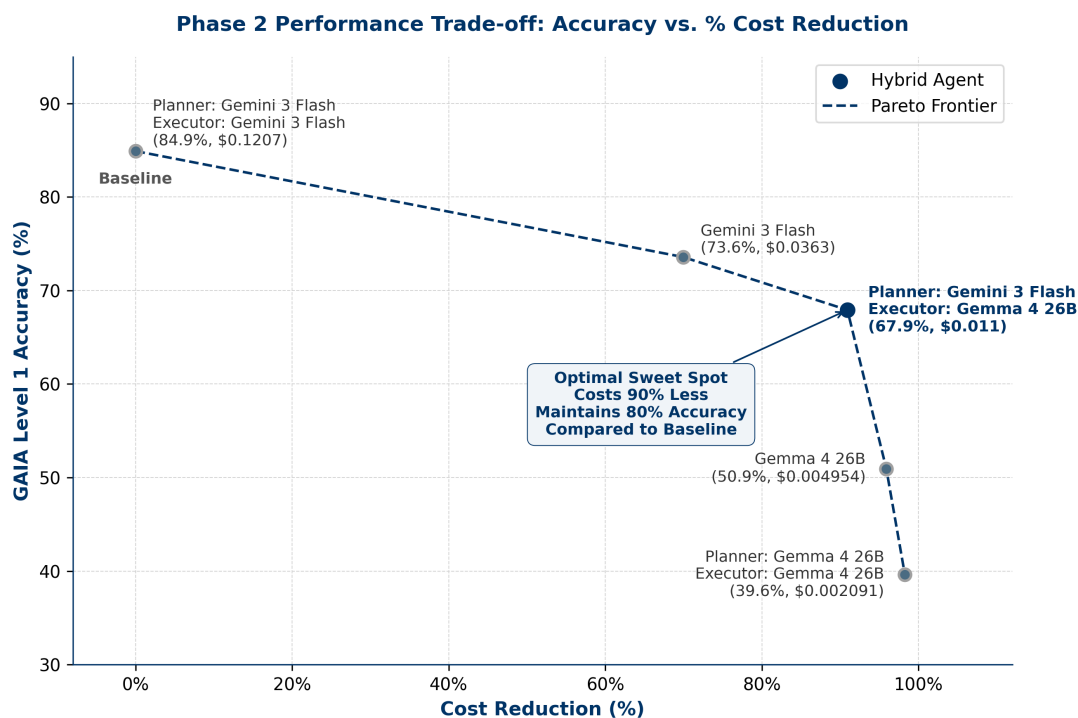


Figure 8.12: Phase 2 accuracy vs. cost-reduction trade-off on GAIA Level 1. The hybrid Planner–Executor is the Pareto-optimal “sweet spot”: ~90% cheaper than the all-commercial baseline while retaining ~80% of its accuracy.

that drives the agent API end-to-end through the vLLM Executor. The serving layer handled this concurrent agentic load using the same FP8 KV configuration validated in Phase 1, closing the loop between the two phases: the optimization measured in Phase 1 is exactly what makes the Phase 2 Executor economically viable.

Chapter 9

CONCLUSION

9.1 Summary of Findings

This project produced the following primary findings from a two-phase study: a systematic serving-optimization benchmarking campaign for vLLM on NVIDIA L4 GPUs (Phase 1), and a GAIA evaluation of a hybrid Planner–Executor agent built on the optimized serving layer (Phase 2):

Finding 1: FP8 quantization is the clear recommendation for production LLM serving. It reduces TTFT P50 by 28–30%, TPOT P50 by 3–35%, and E2E P50 by 6–35% relative to FP16, while causing less than 2.4% quality degradation on ARC-Challenge, MMLU, and GSM8K. The cost per million output tokens drops from \$1.67 to \$1.46 at 16 users and from \$0.68 to \$0.47 at 64 users.

Finding 2: BitsAndBytes (INT4) quantization is counterproductive on the L4 GPU. It doubles TPOT (to 138 ms/tok), increases E2E latency by 181%, and increases cost per million tokens by 93% relative to FP16. This is because the L4 lacks native INT4 compute, and the BitsAndBytes dequantization overhead exceeds the memory bandwidth savings.

Finding 3: Higher concurrency is the dominant cost-reduction lever. Moving from 16 to 64 concurrent users reduces cost per million tokens from \$1.67 to \$0.68 for the FP16 baseline (a 2.5× reduction)—larger than any single parameter change. FP8 at u=64 achieves \$0.47/M tokens, 6.85× cheaper than BitsAndBytes at u=16.

Finding 4: Reasoning models are significantly more expensive to serve. DeepSeek-R1’s E2E P50 (71,382 ms) is 70% higher than Qwen2.5’s (41,955 ms) at identical concurrency and precision, purely because of chain-of-thought output tokens. Applications that do not require step-by-step reasoning should prefer non-reasoning models.

Finding 5: TTFT is insensitive to input prompt length within the 4096-

token window. Across all I/O profiles tested (short inputs of ≈ 15 tokens to long inputs of ≈ 140 tokens), TTFT P50 stays in the 185–195 ms range at 16 users. The L4’s prefill throughput does not become a bottleneck for prompt lengths below several thousand tokens.

Finding 6: E2E latency is driven by output length, not input length. Short-input prompts finish $3.4\times$ faster than long-input prompts, but this is because longer prompts elicit longer reasoning chains, not because of prefill overhead.

Finding 7: Prefix caching provides up to 55% TTFT reduction for shared-prefix workloads. At 8 concurrent users with a shared system prompt, enabling prefix caching reduces TTFT P50 by 53.7–55.7%. The benefit is negligible at $u=1$ (cold cache) and grows with concurrency. This makes prefix caching essential for production chatbot deployments where a system prompt is repeated in every request.

Finding 8: KV cache is not a bottleneck for 7B models on 24 GB GPUs. KV cache utilization peaked at 5.4% across all tested configurations. Parameters like `gpu-memory-utilization` have no measurable effect in this regime. This would change significantly with larger models or longer context windows.

Finding 9: An FP8 KV cache is the most impactful high-concurrency lever. On Qwen3-8B, switching from the default cache to an FP8 KV cache cut TTFT at u_{64} from 30 seconds to 405 ms, tripled throughput at u_{128} ($313 \rightarrow 901$ tok/s), and lowered cost by 65% ($\$1.40 \rightarrow \$0.49/\text{M}$)—all with no measurable quality loss. FP8 KV became the default configuration for the rest of the project.

Finding 10: Aggressive KV-cache compression (TurboQuant) does not pay off on the L4. Across five presets up to 3-bit keys+values, VRAM savings were negligible (~ 400 MiB of 20.6 GiB), throughput fell at every concurrency level, and TPOT regressed by 28–106%, while quality stayed within ± 2.5 pp. The only advantage was faster first-token admit-time under saturation. The technique would likely help on larger models or hardware with cheaper dequant paths, but FP8 KV remains the best operating point here.

Finding 11: Model choice is a real trade-off. Qwen3-8B offers far higher peak throughput (3,599 tok/s, $\$0.12/\text{M}$) and decisively higher quality than Gemma 4 E4B, while Gemma 4 is dramatically more stable under extreme concurrency (525 ms TTFT at u_{128} vs. Qwen3’s 36 s). The right choice depends on whether the workload prioritizes throughput and quality or tail-latency stability.

Finding 12: The hybrid Planner–Executor agent is the Pareto-optimal architecture. On GAIA Level 1, pairing a commercial Planner (Gemini 3 Flash) with a self-hosted Gemma 4 Executor achieved 67.9% accuracy— $\sim 80\%$ of the all-commercial ceiling (84.9%)—at $\$0.0110$ per question, about one-tenth of the all-commercial cost. Planning quality must be bought from the frontier model; execution can be cheaply self-hosted. A fully self-hosted two-model agent (`small-planner-executor`) regressed

to 39.6%, showing that a weak planner is worse than none.

9.2 Practical Recommendations

Table 9.1: Configuration recommendations for different goals.

Goal	Recommendation
Lowest latency	FP8 quantization + high concurrency (u=64+)
Lowest cost per token	FP8 + u=64 (\$0.47/M tokens on L4)
Best quality	FP16 baseline (marginal gain over FP8, max 2.4%)
Best quality/latency/cost balance	FP8 — Pareto-optimal across all benchmarks
Chatbot with system prompts	Enable prefix caching (up to 55% TTFT reduction)
High-concurrency serving	Enable FP8 KV cache (up to 3 $\times$ throughput, no quality loss)
Tail-latency-critical workload	Gemma 4 (sub-600 ms TTFT even at u=128)
Cost-efficient agent	Hybrid Planner-Executor (commercial Planner + self-hosted Executor)
Avoid	BitsAndBytes; TurboQuant on L4; a fully self-hosted two-model agent

9.3 Limitations

- (i) All experiments used a single GPU (L4). Results may differ on A100, H100, or multi-GPU setups.
- (ii) The maximum concurrency tested was 64 users. At higher concurrency, KV cache exhaustion and queuing effects not observed here may emerge.
- (iii) Test duration was 90 seconds per sub-test. Longer tests would better characterize steady-state behavior and thermal throttling effects.
- (iv) Requests are single-turn; multi-turn conversations create different KV cache reuse patterns.
- (v) The custom prompt dataset (37 prompts) is small and may not fully represent production workload diversity.
- (vi) BitsAndBytes testing was limited to u=16; u=64 data was not available due to infrastructure issues.

- (vii) The Phase 2 agent was evaluated only on GAIA Level 1 (53 questions); Levels 2 and 3 and larger sample sizes would tighten the accuracy estimates and stress harder multi-hop reasoning.
- (viii) Phase 2 cost figures depend on an imputed price for self-hosted tokens derived from Phase 1 throughput; the absolute cost ratio would shift with a different GPU rate or utilization assumption.

9.4 Future Work

- (i) **Dynamic task routing:** Implement a lightweight router that sends simple queries directly to the self-hosted Executor, bypassing the Planner API entirely to further reduce cost while reserving the commercial Planner for genuinely complex tasks.
- (ii) **Executor fine-tuning:** Distill the Planner’s reasoning patterns and tool-use traces into the local Executor (Gemma 4) to close the accuracy gap between the hybrid and the all-commercial agent.
- (iii) **Hardware scaling:** Re-run the aggressive KV-cache compression (TurboQuant) and FP8 experiments on newer GPUs (H100/H200), where native FP8/INT4 acceleration should eliminate the dequantization compute bottleneck observed on the L4.
- (iv) Extend the experiment matrix to speculative decoding, which can further reduce TPOT by using a small draft model to propose tokens verified by the main model in batch.
- (v) Evaluate multi-GPU tensor parallelism (TP=2, TP=4) and longer context workloads (8K–128K tokens), where KV cache management becomes the dominant bottleneck and KV compression may finally pay off.
- (vi) Broaden the agent evaluation to GAIA Levels 2 and 3 and additional agent benchmarks, and integrate the serving pipeline with the MLCommons MLPerf Inference suite for standardized cross-system comparison.

Bibliography

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Sarathi-Serve: Taming throughput-latency tradeoff in LLM inference with sarathi-serve. *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [3] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, et al. Chatbot arena: An open platform for evaluating LLMs by human preference. <https://lmarena.ai>, 2024.
- [4] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization. *arXiv preprint arXiv:2110.02861*, 2021.
- [7] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [8] Leo Gao, Jonathan Tow, Baber Abbasi, et al. A framework for few-shot language model evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>, 2023.
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

- [10] LangChain. LangGraph: Building stateful, multi-actor applications with LLMs. <https://langchain-ai.github.io/langgraph/>, 2024.
- [11] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [12] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- [13] NVIDIA. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference. <https://github.com/NVIDIA/TensorRT-LLM>, 2023.
- [14] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [15] Vijay Janapa Reddi, Christine Cheng, David Kanter, et al. MLPerf inference benchmark. <https://mlcommons.org/benchmarks/inference>, 2020.
- [16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [17] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.
- [18] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, et al. Efficiently programming large language models using SGLang. *arXiv preprint arXiv:2312.07104*, 2023.

Appendix A

vLLM PARAMETER REFERENCE

A.1 Install-Time Parameters

vLLM compiles custom CUDA kernels at install time. The hardware platform (NVIDIA CUDA, AMD ROCm, Intel XPU, CPU-only) and the target GPU compute capabilities (TORCH_CUDA_ARCH_LIST) must be specified at build time. Pre-built pip wheels include support for all common NVIDIA architectures (T4=7.5, A100=8.0, L4=8.9, H100=9.0). Using the Docker image `vllm/vllm-openai:latest` avoids CUDA version mismatch issues.

A.2 Key Runtime Parameters (`vllm serve`)

Table A.1: Key vLLM runtime parameters used in this project.

Parameter	Default	Description
<code>-model</code>	—	HuggingFace model ID or local path
<code>-dtype</code>	auto	Weight precision: half, bfloat16, float16
<code>-max-model-len</code>	from config	Maximum context length in tokens
<code>-quantization</code>	None	Quantization: fp8, bitsandbytes, awq, gptq
<code>-gpu-memory-utilization</code>	0.9	Fraction of GPU VRAM for KV cache

Parameter	Default	Description
<code>-max-num-seqs</code>	128	Max concurrent sequences per batch
<code>-enable-chunked-prefill</code>	auto	Split long prefills to reduce head-of-line blocking
<code>-enable-prefix-caching</code>	auto	Reuse KV blocks for shared prompt prefixes
<code>-tensor-parallel-size</code>	1	Number of GPUs for tensor parallelism
<code>-port</code>	8000	HTTP server port

Appendix B

EXPERIMENT CONFIGURATIONS

B.1 Server Tuning Experiments (Pilot)

Table B.1: Server tuning experiment configurations tested in the pilot study.

Name	Changed Parameter	Concurrency
baseline	— (defaults)	1, 4, 8
max_seqs_8	-max-num-seqs 8	1, 4, 8
max_seqs_32	-max-num-seqs 32	1, 8, 32
gpu_mem_80	-gpu-memory-utilization 0.80	1, 4, 8
gpu_mem_95	-gpu-memory-utilization 0.95	1, 4, 8, 16
chunked_prefill	-enable-chunked-prefill	1, 4, 8
prefix_caching	-enable-prefix-caching	1, 4, 8
chunked_prefill + prefix_caching	both	1, 4, 8, 16

B.2 Quantization Experiments

Table B.2: Quantization experiment configurations.

Name	Model	vLLM Args
baseline	DeepSeek-R1	FP16 (default)
fp8_quantization	DeepSeek-R1	-quantization fp8
bitsandbytes_quantization	DeepSeek-R1	-quantization bitsandbytes
non_reasoning_baseline	Qwen2.5-7B	non-reasoning config
non_reasoning_fp8	Qwen2.5-7B	-quantization fp8
non_reasoning_bitsandbytes	Qwen2.5-7B	-quantization bitsandbytes

Table B.3: KV-cache precision experiment configurations (Qwen3-8B).

Name	-kv-cache-dtype	Concurrency
kv_cache_auto	auto (16-bit)	16, 32, 64, 128
kv_cache_fp8	fp8	16, 32, 64, 128
turboquant_k8v4	turboquant_k8v4	32, 64, 128
turboquant_4bit_nc	turboquant_4bit_nc	32, 64, 128
turboquant_k3v4_nc	turboquant_k3v4_nc	64, 128
turboquant_3bit_nc	turboquant_3bit_nc	32, 64, 128

B.3 KV Cache and TurboQuant Experiments (Qwen3-8B)

B.4 Agent Configurations (Phase 2)

Table B.4: Phase 2 agent configurations evaluated on GAIA Level 1.

MODE	Planner	Executor
large-planner-executor	Gemini 3 Flash	Gemini 3 Flash
planner-executor (hybrid)	Gemini 3 Flash	Gemma 4 26B (AWQ, vLLM)
planner-only	Gemini 3 Flash	—
single-agent	—	Gemma 4 26B (AWQ, vLLM)
small-planner-executor	Gemma 4 26B	Gemma 4 26B (AWQ, vLLM)

The self-hosted Executor model is `cyankiwi/gemma-4-26B-A4B-it-AWQ-4bit`, served by vLLM with an FP8 KV cache. The commercial Planner is `gemini-3-flash-preview` via Vertex AI. Tools run in **real** mode for GAIA evaluation (live web search, scraping, document parsing, and E2B-sandboxed Python execution).

Appendix C

DATASET DESCRIPTIONS

C.1 Custom Prompt Dataset

The custom dataset (`dataset.json`) contains 37 prompts across 4 user profiles and 3 length categories:

Table C.1: Custom dataset composition.

Profile	Short	Medium	Long	Total
Coding	5	5	5	15
Creative	5	5	5	15
Reasoning	5	5	5	15
General	5	5	5	15
Total	20	20	20	60

Short prompts are approximately 10–20 tokens; medium prompts approximately 50–62 tokens; long prompts approximately 125–148 tokens.

C.2 Prefix Caching Dataset

The prefix caching dataset (`prefix_dataset.json`) contains 20 prompts across 3 domain-specific system prompts (code reviewer, database consultant, MLOps mentor). Each prompt is paired with a long, detailed system prompt (approximately 250–350 tokens) to maximize prefix reuse. This setup simulates real-world chatbot and API gateway deployments where the same system prompt is sent with every user request.

C.3 Quality Evaluation Datasets

- **ARC-Challenge** (5-shot): 1,172 grade-school science questions requiring reasoning beyond simple fact retrieval. Metric: normalized accuracy.

- **MMLU** (5-shot): 14,042 questions across 57 subjects spanning STEM, humanities, social sciences, and professional domains. Metric: accuracy.
- **GSM8K** (5-shot): 1,319 grade school math word problems. Metric: exact match (strict). Limited to 1,000 samples per evaluation run.

C.4 GAIA Benchmark (Phase 2)

The Phase 2 agent is evaluated on the **GAIA** validation set [12], a benchmark of real-world questions for general AI assistants that require multi-step tool use (web search, file download and parsing, and code execution) to answer. This project uses the **Level 1** split (53 questions), the least difficult tier, which already demands correct tool orchestration. Answers are graded by an exact-match scorer that normalizes case, whitespace, numbers, and currency/unit symbols. Access to the dataset is gated on HuggingFace and requires authentication.